

**THIRD SEMESTER M.Sc COMPUTER SCIENCE
WITH SPECIALISATION IN MACHINE LEARNING**

(2024-2026 BATCH)

LAB RECORD(I)

CML-CC-524- MACHINE INTELLIGENCE LABORATORY

*Submitted to the University of Kerala in partial fulfilment of the requirement for the
award of Degree of Master of Science in Computer Science (with specialization in
Machine Learning)*



Submitted by
Imran Nizam
94824607007

**Department of Computer Science,
University of Kerala, Kariavattom,
Thiruvananthapuram, Kerala, 695581**

1. Cycle No: 1	2. Experiment No: 1
2. Problem	
Read an image and convert it into gray scale image without using built-in function.	
3. Aim and Objectives	
To demonstrate how to convert a color image to grayscale manually, without using any built-in functions for grayscale conversion. It implements the weighted average method to calculate the grayscale value for each pixel and then displays both the original and the converted grayscale images.	
4 Methodology	
• A Python function convert_to_grayscale_manual is defined • The above function takes an image path, opens the image using Pillow, converts it to a NumPy array, and then iterates through each pixel. • For each pixel, it calculates the grayscale value using the formula $0.2989 * R + 0.5870 * G + 0.1140 * B$ • It applies the above values to all three color channels (R, G, B) to create a grayscale pixel. • Finally, it converts the modified NumPy array back into an image using Pillow and returns it. • The defined function convert_to_grayscale_manual is called with a specified image path. • Both the original image and the newly generated grayscale image are then displayed using IPython.display.display to visually compare the results.	
5. Implementation Comparisons	
My Code	AI-Generated(Name)
<pre>import numpy as np from PIL import Image def convert_to_grayscale_manual(image_path): img = Image.open(image_path) img_array = np.array(img) grayscale_array = np.zeros_like(img_array, dtype=np.uint8) height, width, _ = img_array.shape for y in range(height): for x in range(width): r, g, b = img_array[y, x] # Calculate grayscale value gray_value = int(0.2989 * r + 0.5870 * g + 0.1140 * b) # grayscale value grayscale_array[y, x] = [gray_value, gray_value, gray_value]</pre>	<pre>import numpy as np def rgb_to_grayscale(image): # Create an empty array for the grayscale image with the same height and width, but single channel grayscale_image = np.zeros((image.shape[0], image.shape[1]), dtype=np.uint8) for i in range(image.shape[0]): for j in range(image.shape[1]): # Extract R, G, B values R = image[i, j, 0] G = image[i, j, 1] B = image[i, j, 2] # Calculate luminance or simple average gray_value = int((R + G + B) / 3) # Assign to grayscale image grayscale_image[i, j] = gray_value return grayscale_image</pre>

<pre> # NumPy to Image grayscale_img = Image.fromarray(grayscale_array) return grayscale_img from IPython.display import display image_path = '/content/ducati-panigale-6_1730973931548_17617371 03438.avif' try: # image to grayscale grayscale_image = convert_to_grayscale_manual(image_path) # original and grayscale images print("Original Image:") display(Image.open(image_path)) print("\nGrayscale Image:") display(grayscale_image) except FileNotFoundError: print(f"Error: The file '{image_path}' was not found. Please make sure the image file is in the correct directory.") except Exception as e: print(f"An error occurred: {e}") </pre>	
Comments on comparisons	
• Compare the mathematical formulas used to calculate the grayscale value for each pixel. The notebook uses a weighted average (luminosity method: $0.2989R + 0.5870G + 0.1140B$), while AI provided function uses a simple average $(R+G+B)/3$. • The notebook's function takes an image file path and returns a Pillow Image object, while AI function expects a NumPy array as input and returns a 2D NumPy array.	
6. Results	
Original Image :	



Grayscale Image :



Verified by (with Date)

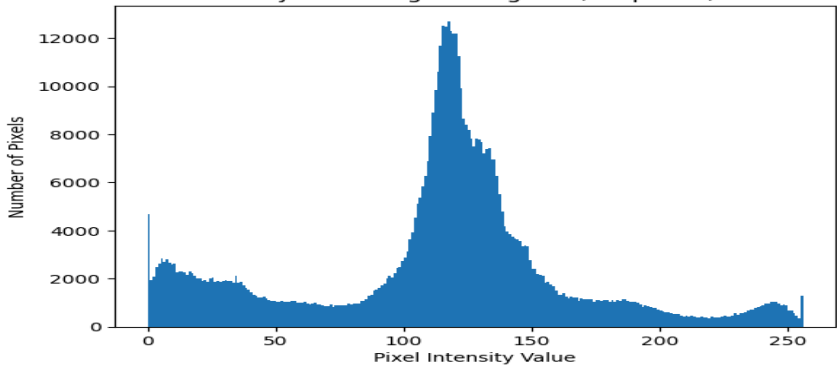
1. Cycle No: 1	2. Experiment No: 2
2. Problem	
Read an image and display the RGB channels of the images separately.	
3. Aim and Objectives	
The main aim and objective of this program is to demonstrate how to read an image and then separate and display its individual Red, Green, and Blue color channels using Python, along with measuring the time taken for this process	
4 Methodology	
- Necessary libraries such as matplotlib.pyplot for plotting and image handling, numpy for numerical operations (though implicitly used by matplotlib for image data), and time for performance measurement are imported. - An image file is read into a NumPy array using plt.imread() - The Red, Green, and Blue color channels are extracted from the loaded image array. Each channel corresponds to a specific slice along the third dimension of the image array. - The original image and its individual RGB channels are displayed in a multi-subplot figure using plt.imshow(). Specific colormaps (Reds, Greens, Blues) are applied to each channel for clear visualization, and titles are added for identification.	
5. Implementation Comparisons	
My Code	AI-Generated(Name)
<pre> from matplotlib import pyplot as plt import numpy as np import time image = plt.imread('/content/ducati-panigale-6_1730973931548_1761737103438.avif') fig,xy = plt.subplots(1,4) start_time = time.time() r_channel = image[:, :,0] g_channel = image[:, :,1] b_channel = image[:, :,2] end_time = time.time() elapsed_time = end_time - start_time xy[0].imshow(image) xy[0].set_title('Original Image') xy[0].axis('off') xy[1].imshow(r_channel, cmap='Reds') </pre>	<pre> import cv2 import matplotlib.pyplot as plt # Read the image image = cv2.imread('path_to_your_image.jpg') # Convert BGR (OpenCV default) to RGB image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB) # Split the channels R, G, B = cv2.split(image_rgb) # Display the channels plt.figure(figsize=(12, 4)) plt.subplot(1, 3, 1) plt.imshow(R, cmap='Reds') plt.title('Red Channel') plt.axis('off') plt.subplot(1, 3, 2) plt.imshow(G, cmap='Greens') plt.title('Green Channel') plt.axis('off') plt.subplot(1, 3, 3) plt.imshow(B, cmap='Blues') plt.title('Blue Channel') plt.axis('off') plt.show() </pre>

<pre> xy[1].set_title('Red Channel') xy[1].axis('off') xy[2].imshow(g_channel, cmap='Greens') xy[2].set_title('Green Channel') xy[2].axis('off') xy[3].imshow(b_channel,cmap = 'Blues') xy[3].set_title('Blue Channel') xy[3].axis('off') plt. show() print(elapsed_time) </pre>	
Comments on comparisons	
• My code uses matplotlib.pyplot.imread() where as the AI generated code uses OpenCV. • While my code plt.imread() directly reads and separates RGB channels using array indexing[:, :,idx], AI snippet's cv2.imread() defaults to BGR and requires explicit cv2.cvtColor(BGR_to_RGB) before splitting channels with cv2.split() to achieve the correct RGB order. •	
6. Results	
Original Image Red Channel Green Channel Blue Channel    	
Verified by (with Date)	

1. Cycle No: 1	2. Experiment No: 3
2. Problem	
Read an image and convert it into binary image using thresholding.	
3. Aim and Objectives	
The aim of this program is to demonstrate how to read an image and convert it into a binary image using thresholding, primarily using the OpenCV library. It shows the steps to load an image, convert it to grayscale, apply a threshold, and then display the original, grayscale, and binary versions of the image.	
4 Methodology	
• Import cv2 for image processing and cv2_imshow from google.colab.patches for displaying images. • Specify the path to an image file and load it using cv2.imread() • Check if the image was loaded successfully; if not, print an error message. • If the loaded image is a color image (has 3 channels), convert it to grayscale using cv2.cvtColor() • Apply binary thresholding to the grayscale image using cv2.threshold(). A threshold value of 127 is used, meaning pixels with intensity above 127 become 255 (white) and those below become 0 (black). • Display Images: Display the original image, the grayscale image, and the resulting binary image using cv2_imshow().	
5. Implementation Comparisons	
My Code	AI-Generated(Name)
<pre>import cv2 from google.colab.patches import cv2_imshow img_path = '/content/ducati-panigale-6_1730973931548_17617371 03438.avif' img = cv2.imread(img_path) if img is None: print(f'Error: Could not read image at {img_path}')</pre> <pre>else: # Convert to grayscale if it's a color image if len(img.shape) == 3: gray_image = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY) else: gray_image = img # It's already grayscale ret, binary_image = cv2.threshold(gray_image, 127, 255, cv2.THRESH_BINARY) # Use the original 'img' for display if it was read successfully, otherwise use gray_image cv2_imshow(img if len(img.shape) == 3 else</pre>	<pre>import cv2 import numpy as np import matplotlib.pyplot as plt # Read the image image = cv2.imread('path_to_your_image.jpg') # Convert to grayscale gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY) # Define threshold value threshold_value = 128 # You can change this value # Apply thresholding binary_image = np.zeros_like(gray_image) binary_image[gray_image > threshold_value] = 255 binary_image[gray_image <= threshold_value] = 0 # Display the binary image plt.imshow(binary_image, cmap='gray') plt.title('Binary Image after Thresholding') plt.axis('off') plt.show()</pre>

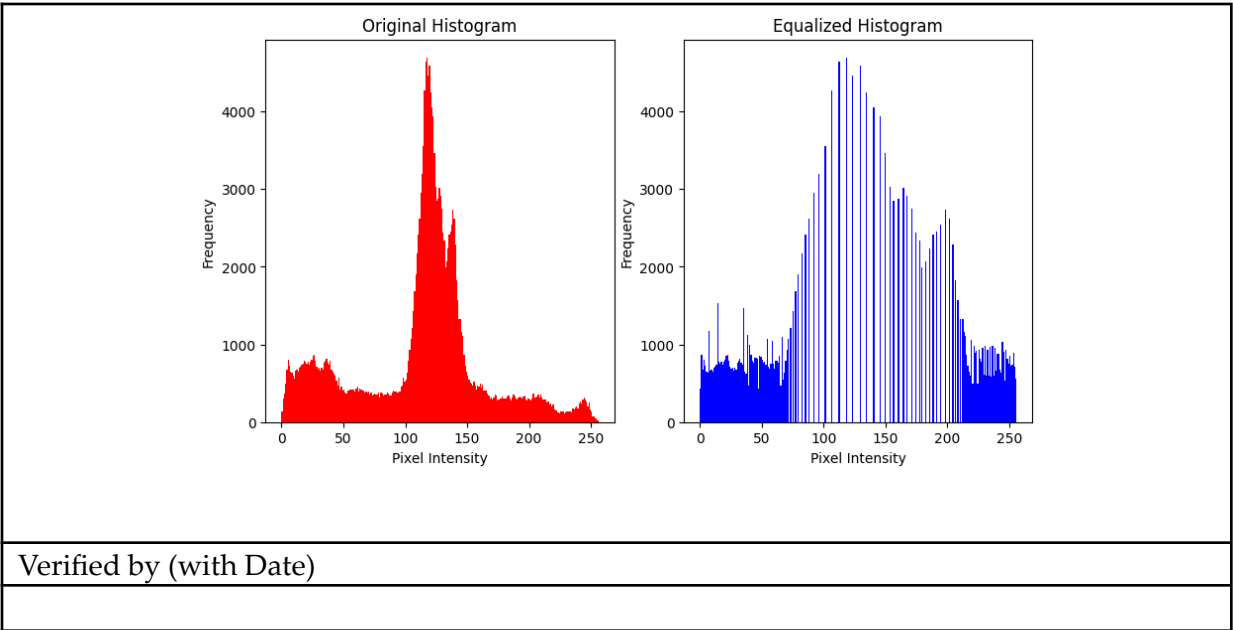
<pre>gray_image) cv2_imshow(gray_image) cv2_imshow(binary_image)</pre>	
Comments on comparisons	
- AI code implements thresholding manually using NumPy array indexing, while my code uses OpenCV's optimized <code>cv2.threshold()</code> function. - AI code uses <code>matplotlib.pyplot.imshow()</code> for displaying images, whereas my code employs <code>google.colab.patches.cv2_imshow()</code> which is a Colab-specific utility.	
6. Results	
	
Verified by (with Date)	

1. Cycle No: 1	2. Experiment No: 4
2. Problem	
Display the histogram of the gray scale image.	
3. Aim and Objectives	
The aim of this program is to demonstrate fundamental image processing operations, specifically focusing on loading an image, converting it to grayscale, displaying the grayscale image, and generating and displaying a histogram of its pixel intensities to understand the distribution of pixel values.	
4 Methodology	
• Essential libraries like OpenCV (cv2) for image processing, NumPy (np) for numerical operations, and Matplotlib (plt) for plotting are imported • A color image is loaded from a predefined file path using OpenCV's imread function. • The loaded color image is then converted into a grayscale image using cv2.cvtColor() • The resulting grayscale image is displayed using Matplotlib, showing the image in a monochromatic scale. • Finally, a histogram of the original color image's pixel intensities is computed and displayed using Matplotlib's hist function, providing insight into the distribution of pixel values across the entire intensity range	
5. Implementation Comparisons	
My Code	AI-Generated(Name)
<pre>import cv2 import numpy as np import matplotlib.pyplot as plt image_path = '/content/ducati-panigale-6_1730973931548_17617371 03438.avif' image = cv2.imread(image_path) plt.hist(image.ravel(), 256, [0, 256]) plt.title('Grayscale Image Histogram (Matplotlib)') plt.xlabel('Pixel Intensity Value') plt.ylabel('Number of Pixels') plt.show() gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY) plt.imshow(gray_image, cmap='gray') plt.title('Grayscale Image') plt.axis('off') plt.show()</pre>	<pre>import cv2 import matplotlib.pyplot as plt # Read the image image = cv2.imread('path_to_your_image.jpg') # Convert to grayscale gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY) # Display the histogram plt.hist(gray_image.ravel(), bins=256, range=[0, 256]) plt.title('Histogram of Grayscale Image') plt.xlabel('Pixel Intensity') plt.ylabel('Frequency') plt.show()</pre>

Comments on comparisons	
• My code uses positional arguments in plt.hist() while the AI uses keyword arguments for clarity. Both plot the histogram for pixel intensities ranging from 0 to 255. • My code assumes image is already a grayscale image, whereas AI explicitly converts it. • The labels and titles differ slightly but serve the same purpose of describing the histogram.	
6. Results	
Grayscale Image Histogram (Matplotlib)  Grayscale Image 	
Verified by (with Date)	

1. Cycle No: 1	2. Experiment No: 5
2. Problem	
Apply histogram equalization on an image and display the resultant image.	
3. Aim and Objectives	
The aim of this program is to demonstrate how to apply histogram equalization on an image and display both the resultant image and its histogram, along with the original image and its histogram for comparison.	
4 Methodology	
- Essential libraries such as cv2 (OpenCV) for image processing, numpy for numerical operations, and matplotlib.pyplot for visualization are imported. - A grayscale image is loaded from a specified path using cv2.imread(). - The cv2.equalizeHist() function is applied to the loaded grayscale image to perform histogram equalization. - Both the original and the histogram-equalized images are displayed side-by-side using matplotlib.pyplot to visually compare the results. - The histograms of both the original and the equalized images are plotted and displayed, illustrating the distribution of pixel intensities before and after equalization.	
5. Implementation Comparisons	
My Code	AI-Generated(Name)
<pre>import cv2 import numpy as np import matplotlib.pyplot as plt image = cv2.imread('/content/ducati-panigale-6_1730973931548_1761737103438.avif', cv2.IMREAD_GRAYSCALE) equalized_image = cv2.equalizeHist(image) plt.figure(figsize=(10, 5)) plt.subplot(1, 2, 1) plt.imshow(image, cmap='gray') plt.title('Original Image') plt.axis('off') plt.subplot(1, 2, 2) plt.imshow(equalized_image, cmap='gray') plt.title('Equalized Image') plt.axis('off') plt.show()</pre>	<pre>import cv2 import matplotlib.pyplot as plt # Load the image in grayscale image = cv2.imread('your_image_path.jpg', cv2.IMREAD_GRAYSCALE) # Apply histogram equalization equalized_image = cv2.equalizeHist(image) # Display original and equalized images side by side plt.figure(figsize=(10, 4)) plt.subplot(1, 2, 1) plt.title('Original Image') plt.imshow(image, cmap='gray') plt.axis('off') plt.subplot(1, 2, 2) plt.title('Equalized Image') plt.imshow(equalized_image, cmap='gray') plt.axis('off') plt.show()</pre>

<pre>plt.figure(figsize=(10, 5)) plt.subplot(1, 2, 1) plt.hist(image.flatten(), 256, [0, 256], color='r') plt.title('Original Histogram') plt.xlabel('Pixel Intensity') plt.ylabel('Frequency') plt.subplot(1, 2, 2) plt.hist(equalized_image.flatten(), 256, [0, 256], color='b') plt.title('Equalized Histogram') plt.xlabel('Pixel Intensity') plt.ylabel('Frequency') plt.show()</pre>	
Comments on comparisons	
My code effectively loads an image, applies histogram equalization, and visualizes the results with side-by-side images and histograms, but it lacks inline comments and explanatory labels that could improve readability and understanding. In contrast, my explanation emphasizes the purpose of each step, such as why histogram equalization is used and how the histograms illustrate the contrast enhancement, providing clearer insights into the process	
6. Results	
Original Image  Equalized Image 	



1. Cycle No: 1	2. Experiment No: 6
2. Problem	
Display the edge map of an image with any edge detection algorithm.	
3. Aim and Objectives	
The main aim of this notebook is to demonstrate edge detection on an image using the Canny algorithm from the OpenCV library, displaying both the original and the edge-detected images	
4 Methodology	
• Import cv2 for image processing and matplotlib.pyplot for plotting. • Convert the loaded color image to grayscale, as many edge detection algorithms work best on single-channel images. • Apply a Gaussian blur filter to the grayscale image to reduce noise, which helps in getting clearer edges. • Utilize the Canny edge detection algorithm (cv2.Canny) on the blurred image with specified threshold values to identify and display the edges. •	
5. Implementation Comparisons	
My Code	AI-Generated(Name)
<pre>import cv2 import matplotlib.pyplot as plt img_path = '/content/2019_Ferrari_812_Superfast_S-A_6.5.jpg' img = cv2.imread(img_path) gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY) plt.imshow(gray, cmap='gray') blurred_img = cv2.GaussianBlur(gray, (5, 5), 0) edges = cv2.Canny(blurred_img, threshold1=50, threshold2=150) plt.figure(figsize=(10, 5)) plt.subplot(1, 2, 1) plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB)) plt.title('Original Image') plt.axis('off') plt.subplot(1, 2, 2) plt.imshow(edges, cmap='gray') plt.title('Canny Edge Map')</pre>	<pre>import cv2 import matplotlib.pyplot as plt # Load the image in grayscale image = cv2.imread('your_image_path.jpg', cv2.IMREAD_GRAYSCALE) # Check if the image was loaded correctly if image is None: raise ValueError("Image not found or unable to load.") # Apply Canny edge detection edges = cv2.Canny(image, threshold1=100, threshold2=200) # Display the original image and the edge map side by side plt.figure(figsize=(10, 5)) plt.subplot(1, 2, 1) plt.title('Original Image') plt.imshow(image, cmap='gray') plt.axis('off') plt.subplot(1, 2, 2) plt.title('Edge Map (Canny)') plt.imshow(edges, cmap='gray') plt.axis('off') plt.show()</pre>

<pre>plt.axis('off') plt.show()</pre>	
Comments on comparisons	
• My code loads the image in color, converts to grayscale for edge detection, and then displays the original in RGB and the AI generated code loads directly in grayscale for simplicity. • My code applies Gaussian blur before edge detection, which can help reduce noise and AI code does not include blurring, but it's often recommended. • My code displays the original in RGB and edges side-by-side with titles. • Both approaches are similar, but your my code correctly converts BGR to RGB for display.	
6. Results	
Original Image  Canny Edge Map 	
Verified by (with Date)	

1. Cycle No: 1	2. Experiment No: 7
2. Problem	
Download any OCR dataset and perform the classification with SVM and KNN. Compare the obtained result	
3. Aim and Objectives	
This notebook aims to perform optical character recognition (OCR) by classifying the MNIST dataset using Support Vector Machine (SVM) and K-Nearest Neighbors (KNN) algorithms. The objective is to compare the classification performance (accuracy) of these two algorithms on the dataset.	
4 Methodology	
• The MNIST dataset, a common OCR dataset, is downloaded using <code>sklearn.datasets.fetch_openml</code> • The pixel values of the images are scaled by dividing by 255.0 to normalize them within a range of 0 to 1. • The dataset is split into training and testing sets using <code>train_test_split</code> with an 80/20 ratio and a <code>random_state</code> for reproducibility. • A Support Vector Machine (SVM) classifier with a linear kernel is initialized and trained on the training data. • Predictions are made on the test set, and the accuracy of the SVM model is calculated using <code>accuracy_score</code>. • A K-Nearest Neighbors (KNN) classifier with <code>n_neighbors=5</code> is initialized and trained on the training data. • Predictions are made on the test set, and the accuracy of the KNN model is calculated. • The accuracies obtained from both the SVM and KNN classifiers are printed for comparison.	
5. Implementation Comparisons	
My Code	AI-Generated(Name)
<pre> from sklearn.datasets import fetch_openml # Download the MNIST dataset mnist = fetch_openml('mnist_784', version=1, as_frame=False) X, y = mnist.data, mnist.target import numpy as np X = X / 255.0 from sklearn.model_selection import train_test_split X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42) from sklearn.svm import SVC from sklearn.metrics import accuracy_score # Initialize and train the SVM classifier # Using a linear kernel for simplicity, other kernels can be explored svm_classifier = SVC(kernel='linear', </pre>	<pre> import time from sklearn.datasets import fetch_openml from sklearn.model_selection import train_test_split from sklearn.preprocessing import StandardScaler from sklearn.svm import SVC from sklearn.neighbors import KNeighborsClassifier from sklearn.metrics import accuracy_score, classification_report # 1. Load a subset of the MNIST dataset (OCR data) print("Loading MNIST data...") mnist = fetch_openml('mnist_784', version=1, as_frame=False, parser='auto') X, y = mnist.data[:10000], mnist.target[:10000] # Using 10k samples for speed # 2. Preprocess: Scale features (SVM is sensitive to scale) scaler = StandardScaler() X_scaled = scaler.fit_transform(X) # 3. Split into training and testing sets X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2, random_state=42) # 4. Support Vector Machine (SVM) Classification print("\n--- Training SVM ---") start_svm = time.time() svm_clf = SVC(kernel='rbf', C=10, gamma='scale') # RBF kernel is standard for non-linear OCR </pre>

<pre> random_state=42) svm_classifier.fit(X_train, y_train) # Make predictions and evaluate accuracy y_pred_svm = svm_classifier.predict(X_test) accuracy_svm = accuracy_score(y_test, y_pred_svm) print(f'SVM Accuracy: {accuracy_svm}') from sklearn.neighbors import KNeighborsClassifier knn_classifier = KNeighborsClassifier(n_neighbors=5) knn_classifier.fit(X_train, y_train) y_pred_knn = knn_classifier.predict(X_test) accuracy_knn = accuracy_score(y_test, y_pred_knn) print(f'KNN Accuracy: {accuracy_knn}') </pre>	<pre> svm_clf.fit(X_train, y_train) svm_time = time.time() - start_svm svm_preds = svm_clf.predict(X_test) # 5. K-Nearest Neighbors (KNN) Classification print("--- Training KNN ---") start_knn = time.time() knn_clf = KNeighborsClassifier(n_neighbors=3) # Typical k value for MNIST knn_clf.fit(X_train, y_train) knn_time = time.time() - start_knn knn_preds = knn_clf.predict(X_test) # 6. Compare Results print("\nRESULTS COMPARISON:") print(f'SVM Accuracy: {accuracy_score(y_test, svm_preds):.4f} (Time: {svm_time:.2f}s)') print(f'KNN Accuracy: {accuracy_score(y_test, knn_preds):.4f} (Time: {knn_time:.2f}s)') print("\nSVM Detail:") print(classification_report(y_test, svm_preds)) </pre>
Comments on comparisons - While my code uses a simple Min-Max scaling while AI's approach uses StandardScaler, which typically helps SVMs converge faster by centering the pixel data. You opted for a linear kernel, which is faster but often less accurate for handwritten digits than the RBF kernel I suggested for capturing non-linear shapes. - Finally, My script processes the full dataset, which provides higher precision but will take significantly longer to train than AI's 10,000-sample subset.	
6. Results	
SVM Accuracy: 0.9351428571428572 KNN Accuracy: 0.9700714285714286	
Verified by (with Date)	

1. Cycle No: 1	2. Experiment No: 8
2. Problem	
Implement any two segmentation algorithms and compare the efficiency with ground truth.	
3. Aim and Objectives	
The aim of this notebook is to implement and compare the efficiency of two segmentation algorithms with ground truth. Currently, the code demonstrates image segmentation using the Watershed algorithm.	
4 Methodology	
• The input image is converted to grayscale and then subjected to Otsu's thresholding to obtain a binary image. • Morphological opening operation is applied to the binary image to remove noise. • Sure Background is identified by dilating the binary image. • Sure Foreground is identified by applying a distance transform to the binary image and then thresholding the result. • Unknown Region is determined by subtracting the sure foreground from the sure background • Marker-based Segmentation : ○ Connected components are found in the sure foreground to create initial markers. ○ These markers are adjusted to properly label the background and unknown regions. ○ The Watershed algorithm (cv2.watershed) is then applied using these markers and the original image to perform the segmentation	
5. Implementation Comparisons	
My Code	AI-Generated(Name)
<pre>import cv2 import numpy as np import numpy as np from IPython.display import Image, display from matplotlib.pyplot as plt def imshow(img, ax=None): if ax is None: ret, encoded = cv2.imencode(".jpg", img) display(Image(encoded)) else: ax.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB)) ax.axis('off')</pre>	<pre>import cv2 import numpy as np import time # 1. Otsu's Thresholding (Global Statistics) def otsu_segmentation(image): gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY) start_time = time.time() # cv2.THRESH_OTSU automatically calculates the optimal threshold _, mask = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU) exec_time = time.time() - start_time return mask, exec_time # 2. K-Means Clustering (Feature Space) def kmeans_segmentation(image, k=2): pixel_values = image.reshape((-1, 3)).astype(np.float32) criteria = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 10, 1.0) start_time = time.time() _, labels, centers = cv2.kmeans(pixel_values, k, None, criteria, 10, cv2.KMEANS_RANDOM_CENTERS) exec_time = time.time() - start_time</pre>

<pre> #Image loading img = cv2.imread("/content/ducati-panigale-6_1730973931 548_1761737103438.avif") # Show image imshow(img) gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY) imshow(gray) #Threshold Processing ret, bin_img = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU) imshow(bin_img) kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (3, 3)) bin_img = cv2.morphologyEx(bin_img, cv2.MORPH_OPEN, kernel, iterations=2) imshow(bin_img) # Create subplots with 1 row and 2 columns fig, axes = plt.subplots(nrows=2, ncols=2, figsize=(8, 8)) # sure background area sure_bg = cv2.dilate(bin_img, kernel, iterations=3) imshow(sure_bg, axes[0,0]) axes[0, 0].set_title('Sure Background') </pre>	<pre> # Convert to binary mask for comparison centers = np.uint8(centers) segmented_data = centers[labels.flatten()] segmented_image = segmented_data.reshape(image.shape) gray_segmented = cv2.cvtColor(segmented_image, cv2.COLOR_BGR2GRAY) _, mask = cv2.threshold(gray_segmented, 127, 255, cv2.THRESH_BINARY) return mask, exec_time </pre>
--	--

```
# Distance transform

dist = cv2.distanceTransform(bin_img,
cv2.DIST_L2, 5)

imshow(dist, axes[0,1])

axes[0, 1].set_title('Distance Transform')


#foreground area

ret, sure_fg = cv2.threshold(dist, 0.5 * dist.max(),
255, cv2.THRESH_BINARY)

sure_fg = sure_fg.astype(np.uint8)

imshow(sure_fg, axes[1,0])

axes[1, 0].set_title('Sure Foreground')

# unknown area

unknown = cv2.subtract(sure_bg, sure_fg)

imshow(unknown, axes[1,1])

axes[1, 1].set_title('Unknown')

plt.show()

# Marker labelling

# sure foreground

ret, markers = cv2.connectedComponents(sure_fg)

# Add one to all labels so that background is not 0,
but 1

markers += 1

# mark the region of unknown with zero

markers[unknown == 255] = 0

fig, ax = plt.subplots(figsize=(6, 6))

ax.imshow(markers, cmap="tab20b")

ax.axis('off')

plt.show()
```

<pre> # watershed Algorithm markers = cv2.watershed(img, markers) fig, ax = plt.subplots(figsize=(5, 5)) ax.imshow(markers, cmap="tab20b") ax.axis('off') plt.show() labels = np.unique(markers) bike = [] for label in labels[2:]: # Create a binary image in which only the area of the label is in the foreground target = np.where(markers == label, 255, 0).astype(np.uint8) # Perform contour extraction on the created binary image contours, hierarchy = cv2.findContours(target, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE) bike.append(contours[0]) # Draw the outline img = cv2.drawContours(img, bike, -1, color=(0, 23, 223), thickness=2) imshow(img) </pre>	
Comments on comparisons - AI Generated (Otsu's): Simply finds one brightness value (threshold) that best separates the histogram into two groups. It treats every pixel independently. - My Code (Watershed): Treats the image like a topographic map. It uses the "Distance Transform" to find local peaks (the "seeds" or "catchment basins") and then simulates "flooding" the image from those seeds until the regions meet at boundaries. - AI Code: Has no concept of "blobs" or "shapes." If a background pixel happens to be dark like the bike, it gets labeled as a bike. - My Code: Uses Morphological Operations (dilate, distanceTransform) to determine what is definitely	

background and definitely foreground. This creates a much more "solid" object segmentation.

- AI Code is significantly faster because it only looks at the image histogram.
- My code is slower because it requires multiple passes: thresholding, distance transforms, connected components, and then the watershed "flooding" process.

6. Results



Noise Removal



Sure Background



Distance Transform



Sure Foreground



Unknown



Marker Labelling



Verified by (with Date)

1. Cycle No: 1	2. Experiment No: 9
2. Problem	
Input an image and perform the following morphological operations i) Dilation ii) Erosion iii) Opening iv) Closing Display the results.	
3. Aim and Objectives	
The aim of this notebook is to demonstrate and visualize different morphological operations on an input image using the OpenCV library. Specifically, it performs dilation, erosion, opening, and closing operations and displays their results.	
4 Methodology	
- Essential libraries such as cv2 (OpenCV) for image processing, numpy for numerical operations, and matplotlib.pyplot for displaying images are imported. - An image named '314.png' is loaded into the notebook using cv2.imread(). - A 5x5 kernel of ones is created using numpy.ones(). This kernel is crucial for the morphological operations. - Performing Morphological Operations: The loaded image is then subjected to four different morphological operations using OpenCV functions: - Erosion: cv2.erode() is applied to shrink foreground objects. - Dilation: cv2.dilate() is applied to expand foreground objects. - Opening: cv2.morphologyEx() with cv2.MORPH_OPEN performs an erosion followed by a dilation, useful for removing small objects. - Closing: cv2.morphologyEx() with cv2.MORPH_CLOSE performs a dilation followed by an erosion, useful for closing small holes or gaps. - Displaying Results: Finally, all the original and processed images (erosion, dilation, opening, and closing) are displayed side-by-side using matplotlib.pyplot for visual comparison, with appropriate titles for each image.	
5. Implementation Comparisons	
My Code	AI-Generated(Name)
<pre>import cv2 import numpy as np from matplotlib import pyplot as plt image = cv2.imread('/content/314.png') kernel = np.ones((5,5), np.uint8) #erosion erosion = cv2.erode(image, kernel, iterations=1) #dialation dialation = cv2.dilate(image, kernel, iterations=1) #opening opening = cv2.morphologyEx(image,</pre>	<pre>import cv2 import numpy as np import matplotlib.pyplot as plt # Load and binarize the image (using your Otsu's result as input) gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY) _, bin_img = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU) # Define a 5x5 structuring element kernel = np.ones((5, 5), np.uint8) # i) Dilation: Expands foreground boundaries, filling small holes dilation = cv2.dilate(bin_img, kernel, iterations=1) # ii) Erosion: Shrinks foreground boundaries, removing small noise erosion = cv2.erode(bin_img, kernel, iterations=1) # iii) Opening: Erosion followed by Dilation; removes small "islands"</pre>

<pre> cv2.MORPH_OPEN, kernel) #closing closing = cv2.morphologyEx(image, cv2.MORPH_CLOSE, kernel) titles = ['image', 'erosion', 'dialation', 'opening', 'closing'] images = [image, erosion, dialation, opening, closing] for i in range(5): plt.subplot(2, 3, i+1) plt.imshow(images[i], cmap='gray') plt.title(titles[i]) plt.xticks([], plt.yticks([])) plt.show() </pre>	<pre> opening = cv2.morphologyEx(bin_img, cv2.MORPH_OPEN, kernel) # iv) Closing: Dilation followed by Erosion; fills small "gaps" closing = cv2.morphologyEx(bin_img, cv2.MORPH_CLOSE, kernel) # Display Results titles = ['Original Binary', 'Dilation', 'Erosion', 'Opening', 'Closing'] images = [bin_img, dilation, erosion, opening, closing] plt.figure(figsize=(12, 8)) for i in range(5): plt.subplot(2, 3, i+1) plt.imshow(images[i], 'gray') plt.title(titles[i]) plt.axis('off') plt.tight_layout() plt.show() </pre>
--	---

Comments on comparisons

- My Code: Processes the image in its original BGR color format. While OpenCV can technically apply morphology to color images (it processes each channel independently), these operations are traditionally intended for binary (black and white) or grayscale images to analyze shapes.
- AI Code: First converts the image to grayscale and applies Otsu's Thresholding. This creates a binary mask, which is the standard way to use morphology for tasks like removing noise or filling holes in objects
- My Code: Uses `plt.imshow(..., cmap='gray')` on a BGR image. Matplotlib's `imshow` ignores `cmap='gray'` if the input has 3 channels (color), displaying it in color instead. Additionally, since OpenCV reads images as BGR and Matplotlib expects RGB, your colors will appear swapped (e.g., reds looking blue).
- AI Code: Uses a 2D grayscale array, allowing `cmap='gray'` to correctly render a black-and-white result.

6. Results



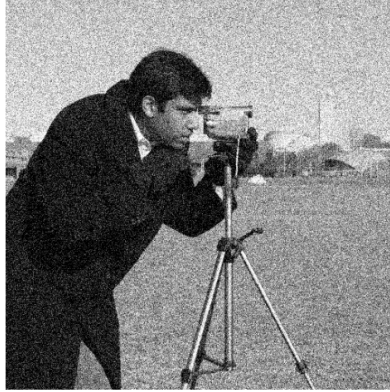
1. Cycle No: 1	2. Experiment No: 10
2. Problem	
Implement any image restoration algorithm.	
3. Aim and Objectives	
The aim of this notebook is to demonstrate an image restoration algorithm, specifically Non-local Means Denoising. It loads a sample image, adds artificial noise to it, applies the denoising algorithm, and then visualizes the original, noisy, and denoised images to show the effectiveness of the algorithm.	
4 Methodology	
• A sample camera image is loaded and converted to a floating-point representation. • Gaussian noise is artificially added to the original image to simulate a noisy scenario. • The standard deviation of the noise present in the noisy image is estimated. • The Non-local Means (NL-means) denoising algorithm is applied to the noisy image, utilizing the estimated noise standard deviation and specific parameters for patch size and search distance. • Finally, the original, noisy, and denoised images are displayed side-by-side to visually assess the effectiveness of the restoration process.	
5. Implementation Comparisons	
My Code	AI-Generated(Name)
<pre>import numpy as np import matplotlib.pyplot as plt from skimage import data, img_as_float from skimage.restoration import denoise_nl_means, estimate_sigma from skimage.util import random_noise # Load a sample image img = img_as_float(data.camera()) # Add some random noise to the image sigma = 0.155 noisy_img = random_noise(img, var=sigma**2) # Estimate the noise standard deviation from the noisy image sigma_est = np.mean(estimate_sigma(noisy_img)) # Apply Non-local Means Denoising denoisy_img = denoise_nl_means(noisy_img,</pre>	<pre>import cv2 import numpy as np import matplotlib.pyplot as plt # 1. Load the image image = cv2.imread('/content/ducati-panigale-6_1730973931548_1761737103438.avif') image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB) # 2. Add Artificial Gaussian Noise noise = np.random.normal(0, 25, image_rgb.shape).astype(np.uint8) noisy_image = cv2.add(image_rgb, noise) # 3. Apply Non-Local Means Denoising # Parameters: h=10 (filter strength), hColor=10, templateWindowSize=7, searchWindowSize=21 denoised_image = cv2.fastNlMeansDenoisingColored(noisy_image, None, 10, 10, 7, 21) # 4. Visualization titles = ['Original', 'Noisy (Gaussian)', 'NLM Denoised'] images = [image_rgb, noisy_image, denoised_image] plt.figure(figsize=(15, 5)) for i in range(3): plt.subplot(1, 3, i+1) plt.imshow(images[i]) plt.title(titles[i]) plt.axis('off')</pre>

<pre> h=1.15 * sigma_est, fast_mode=True, patch_size=5, # 5x5 patches patch_distance=6) # 13x13 search area #results fig, axes = plt.subplots(nrows=1, ncols=3, figsize=(15, 5), sharex=True, sharey=True) ax = axes.ravel() ax[0].imshow(img, cmap=plt.cm.gray) ax[0].set_title('Original image') ax[1].imshow(noisy_img, cmap=plt.cm.gray) ax[1].set_title('Noisy image') ax[2].imshow(denoisy_img, cmap=plt.cm.gray) ax[2].set_title('Denoised image (NL-means)') for a in ax: a.axis('off') fig.tight_layout() plt.show()</pre>	<pre>plt.show()</pre>
Comments on comparisons	
• My Code (scikit-image): Works with floating-point images (0.0 to 1.0). It is highly precise and provides scientific tools like <code>estimate_sigma</code> to automatically calculate how much noise is in the image before cleaning it. • AI Code (OpenCV): Works with 8-bit integers (0 to 255). It is written in C++, making it significantly faster than the scikit-image version, but it requires you to manually set the filtering strength (<code>h</code>). • My Code: Explicitly defines <code>patch_size</code> and <code>patch_distance</code>. This gives you granular control over the "non-local" search. • Your Code: Uses <code>fastNlMeansDenoisingColored</code>, which is a "black box" optimization designed for speed. It processes color channels simultaneously, whereas your current snippet is optimized for grayscale (<code>data.camera()</code>).	
6. Results	

Original image



Noisy image



Denoised image (NL-means)



Verified by (with Date)